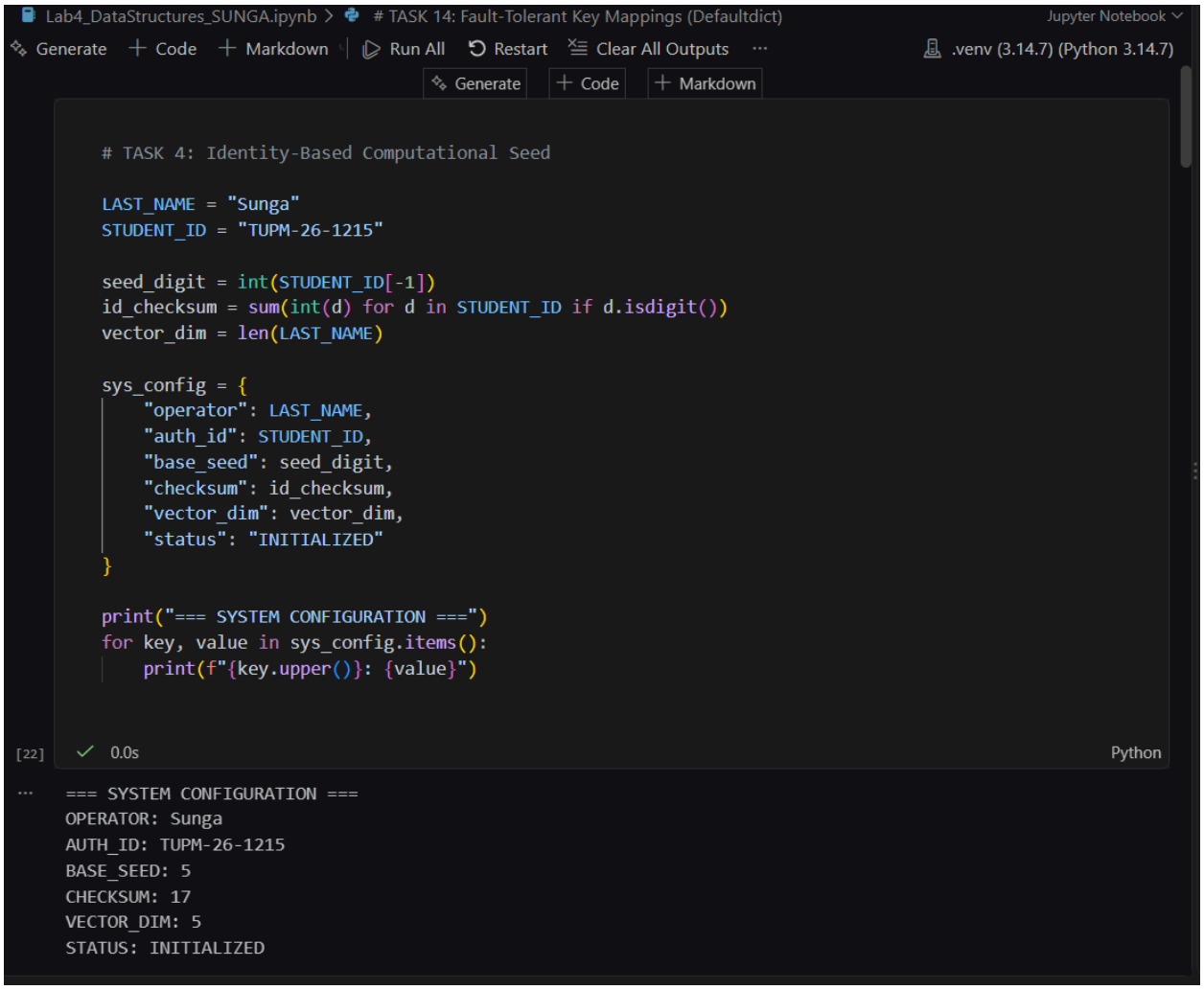
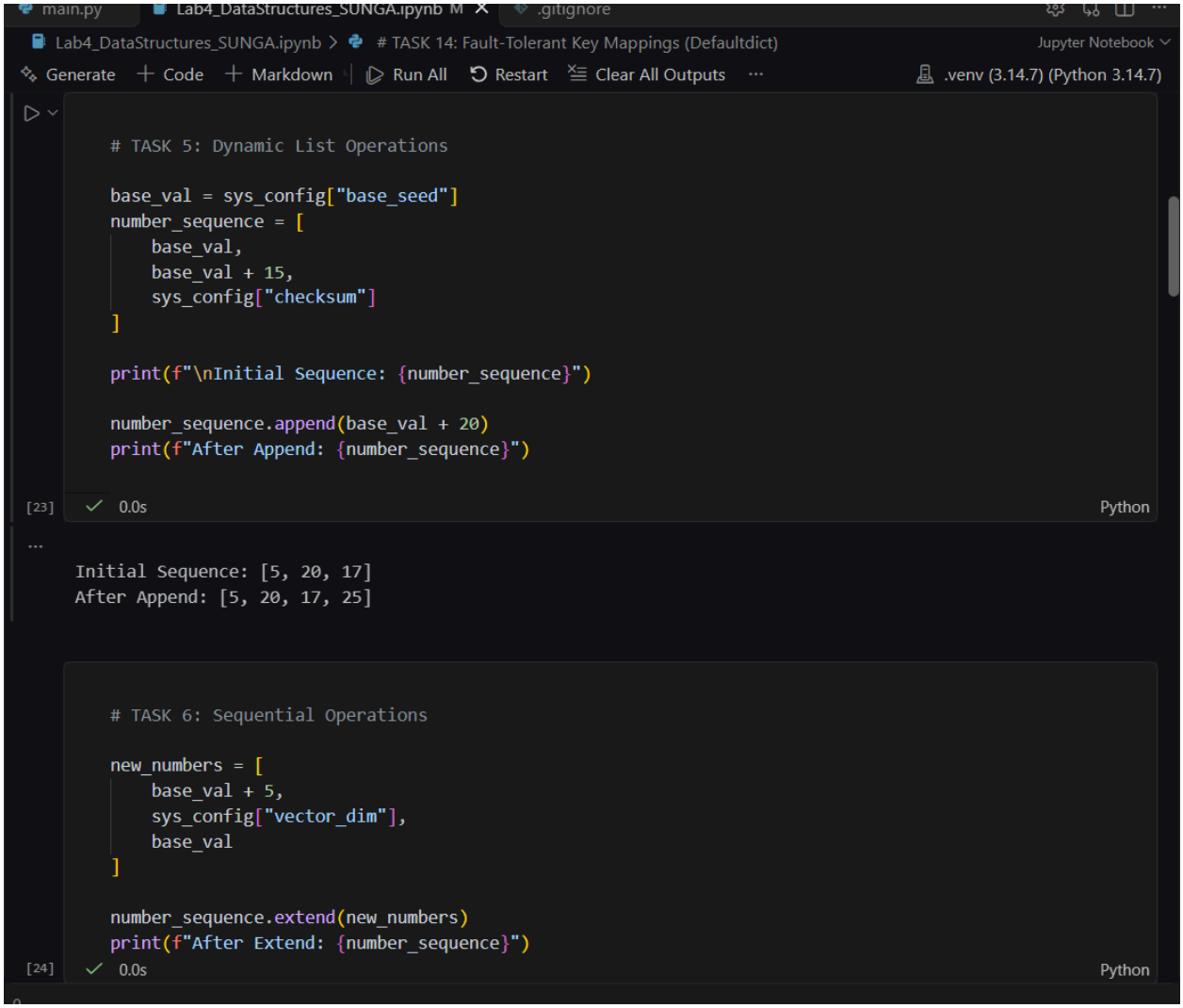
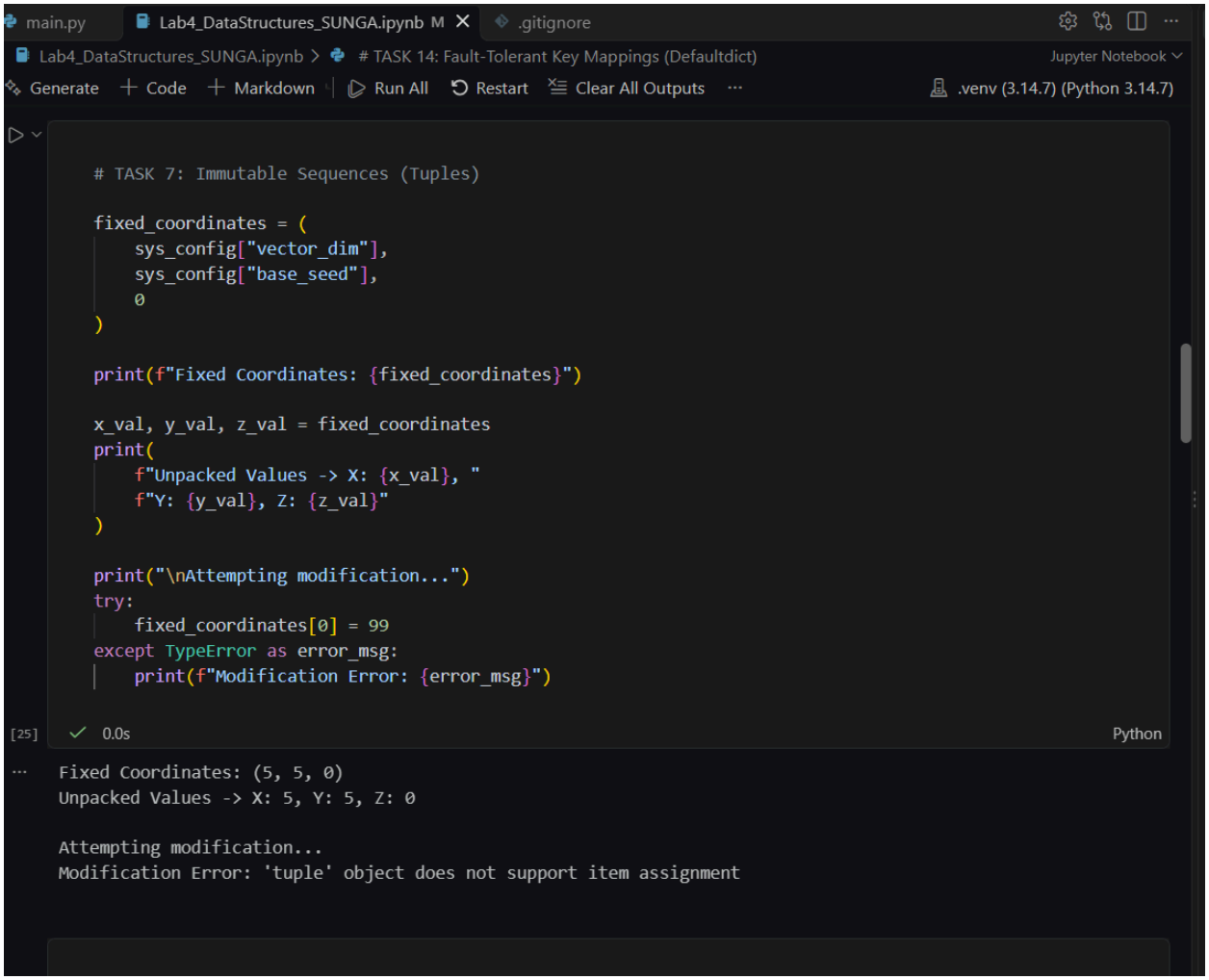
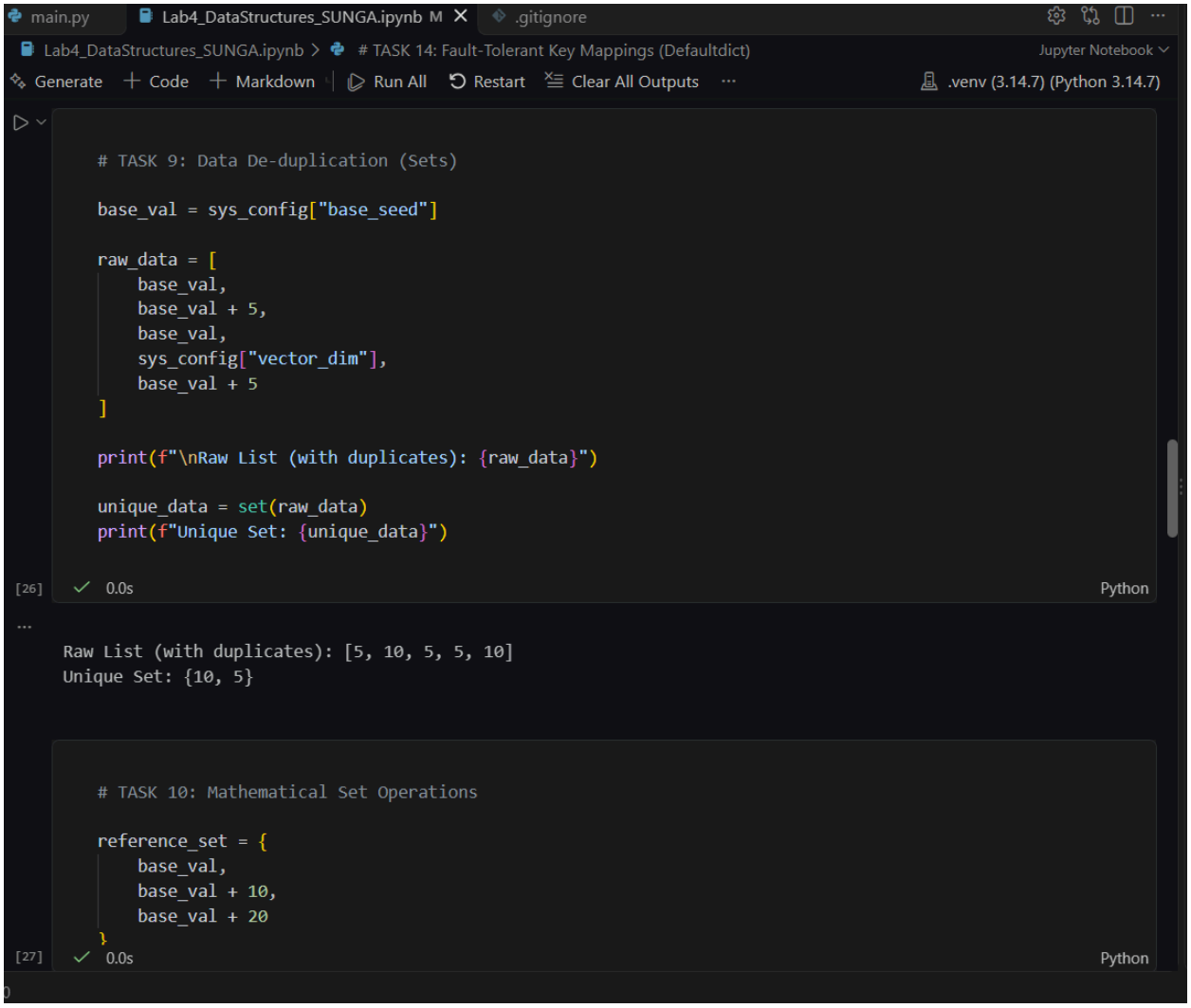
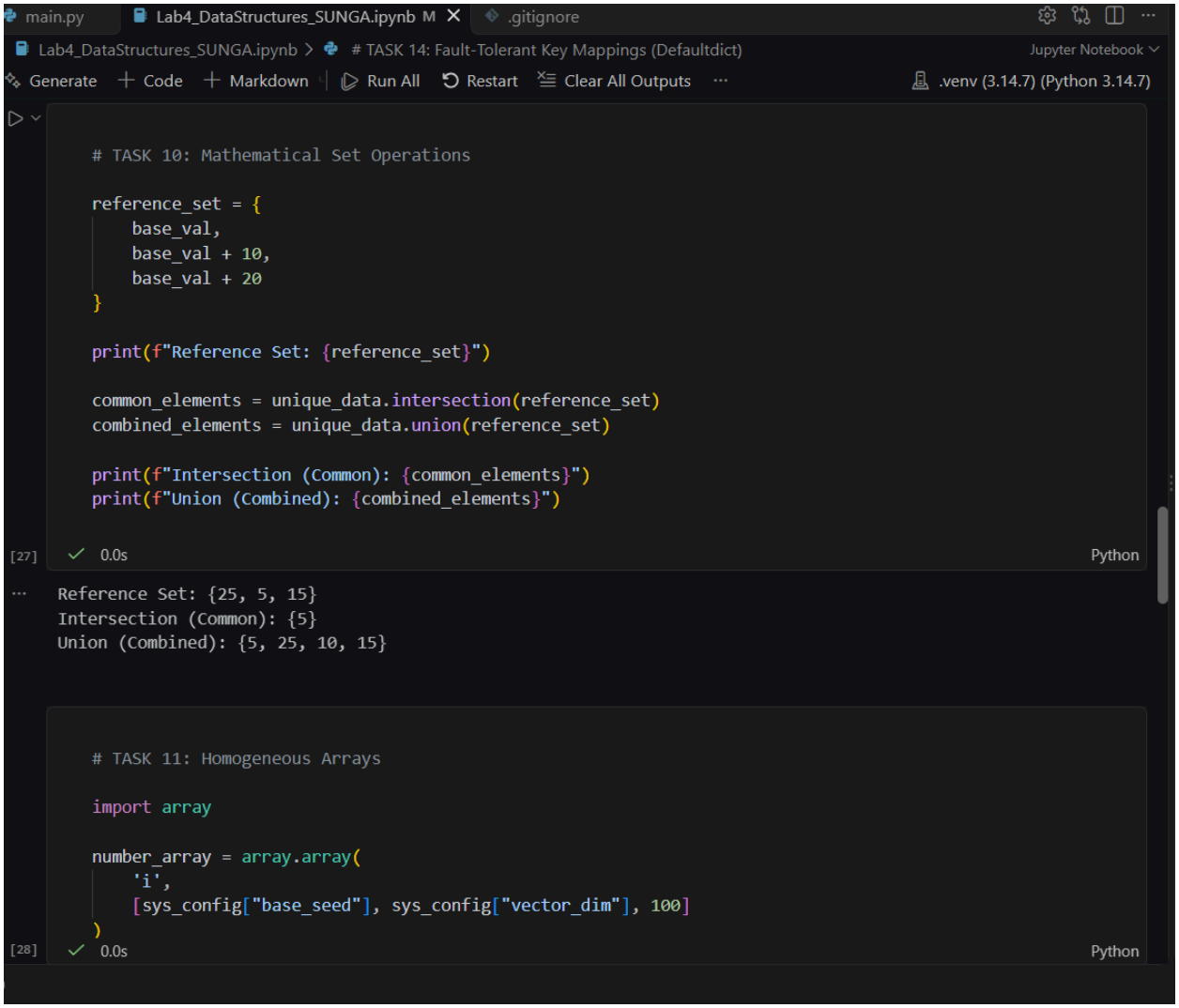
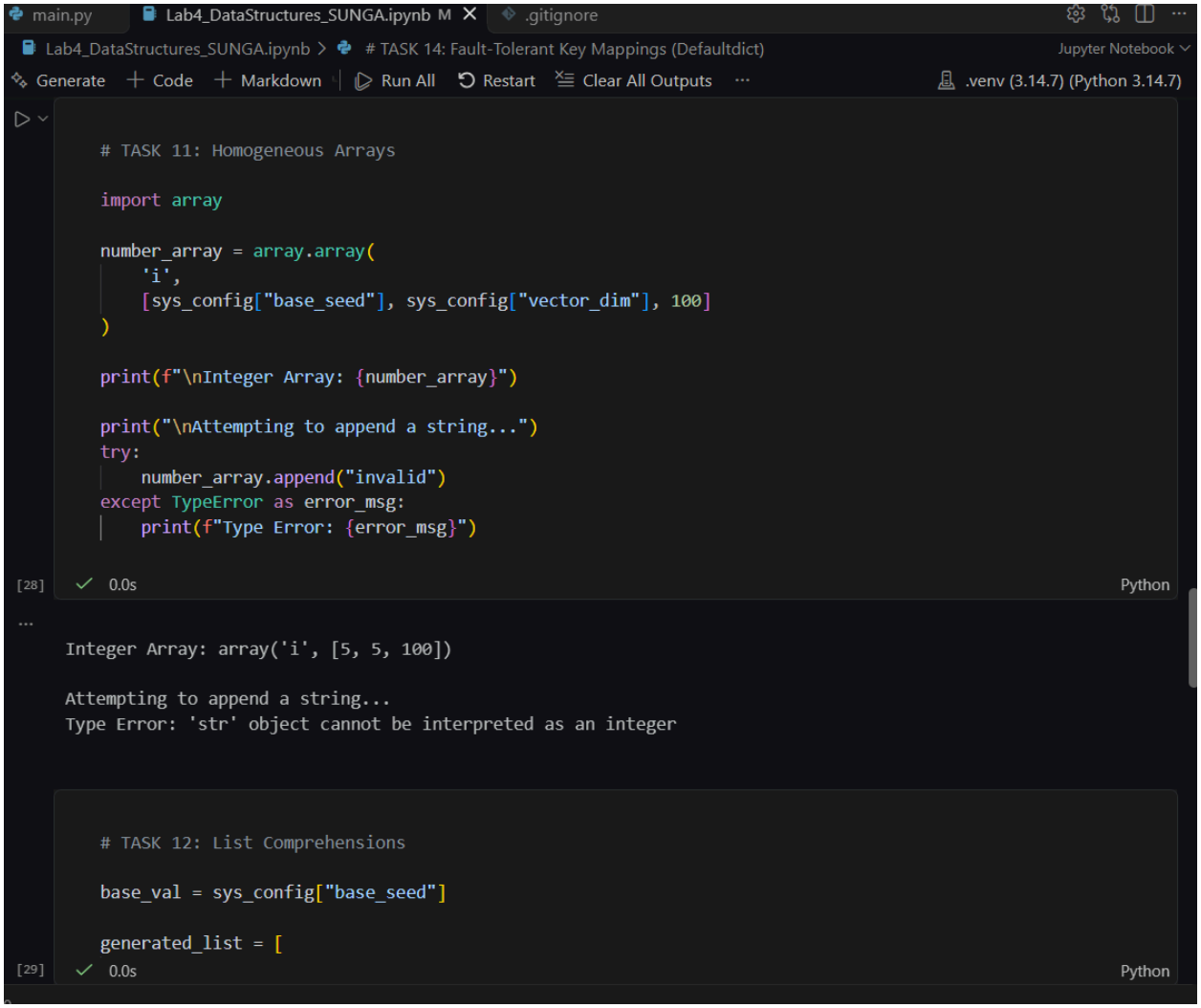

Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
Generate
Code
Markdown
TASK 4: Identity-Based Computational Seed
LAST_NAME = "Sunga"
STUDENT_ID = "TUPM-26-1215"
seed_digit = int(STUDENT_ID[-1])
id_checksum = sum(int(d) for d in STUDENT_ID if d.isdigit())
vector_dim = len(LAST_NAME)
sys_config = {
"operator": LAST_NAME,
"auth_id": STUDENT_ID,
"base_seed": seed_digit,
"checksum": id_checksum,
"vector_dim": vector_dim,
"status": "INITIALIZED"
}
print("=== SYSTEM CONFIGURATION ===")
for key, value in sys_config.items():
print(f"{key.upper()}: {value}")
[22]
0.0s
Python
=== SYSTEM CONFIGURATION ===
OPERATOR: Sunga
AUTH_ID: TUPM-26-1215
BASE_SEED: 5
CHECKSUM: 17
VECTOR_DIM: 5
STATUS: INITIALIZED

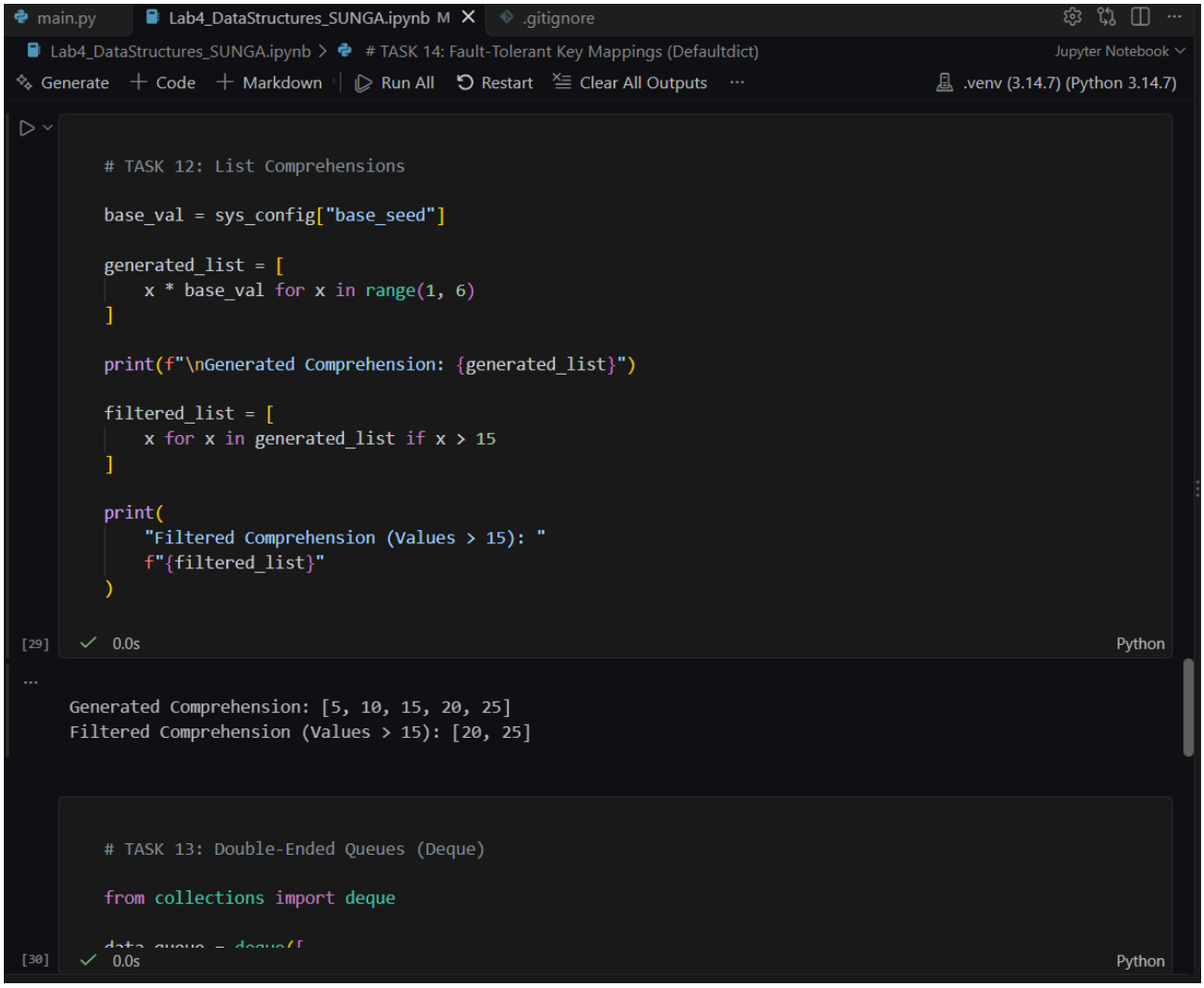

main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
TASK 5: Dynamic List Operations
base_val = sys_config["base_seed"]
number_sequence = [
base_val,
base_val + 15,
sys_config["checksum"]
]
print(f"\nInitial Sequence: {number_sequence}")
number_sequence.append(base_val + 20)
print(f"After Append: {number_sequence}")
[23]
0.0s
Python
Initial Sequence: [5, 20, 17]
After Append: [5, 20, 17, 25]
TASK 6: Sequential Operations
new_numbers = [
base_val + 5,
sys_config["vector_dim"],
base_val
]
number_sequence.extend(new_numbers)
print(f"After Extend: {number_sequence}")
[24]
0.0s
Python

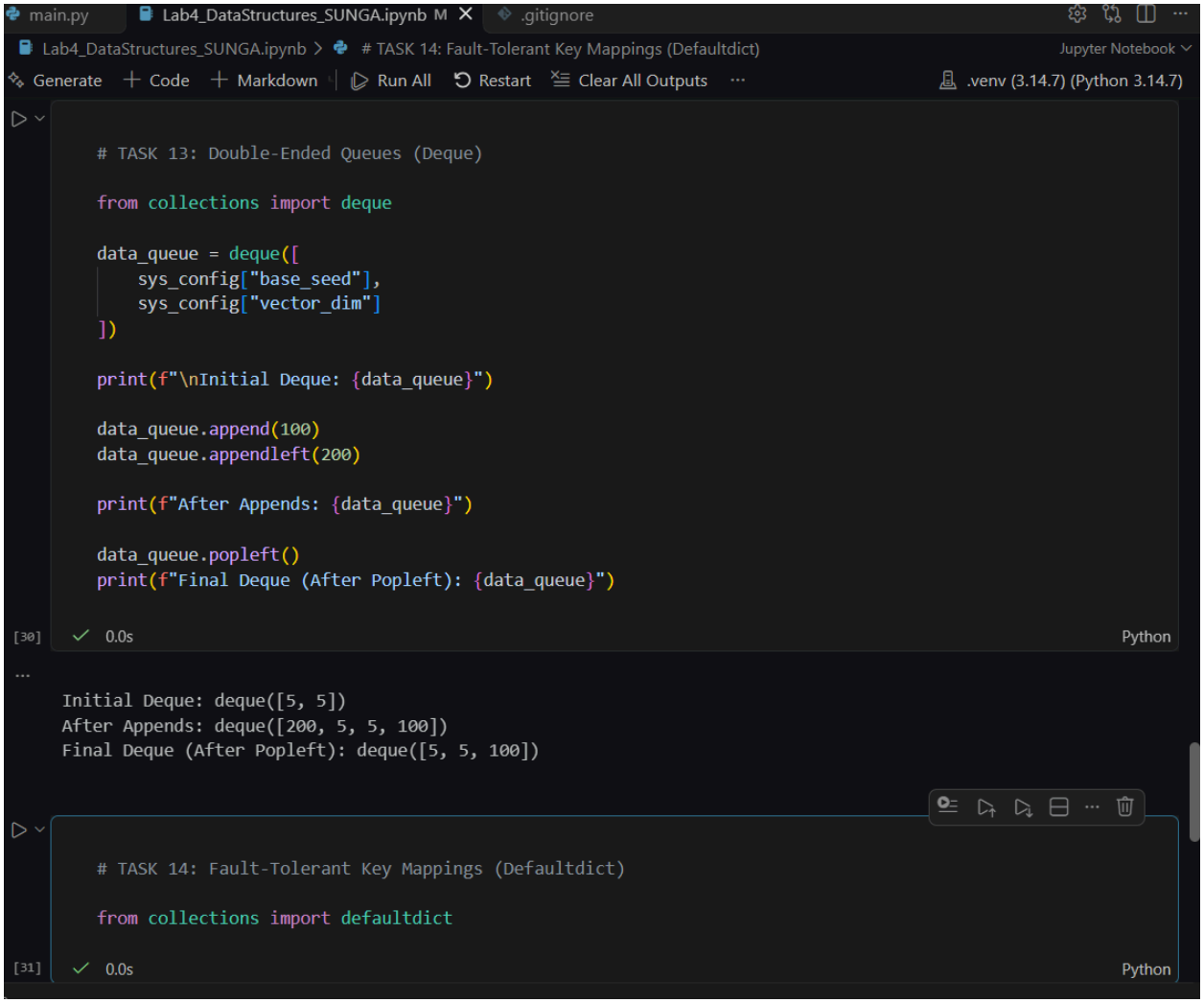

main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
TASK 7: Immutable Sequences (Tuples)
fixed_coordinates = (
sys_config["vector_dim"],
sys_config["base_seed"],
0
)
print(f"Fixed Coordinates: {fixed_coordinates}")
x_val, y_val, z_val = fixed_coordinates
print(
f"Unpacked Values -> X: {x_val}, "
f"Y: {y_val}, Z: {z_val}"
)
print("\nAttempting modification...")
try:
fixed_coordinates[0] = 99
except TypeError as error_msg:
print(f"Modification Error: {error_msg}")
[25]
0.0s
Python
Fixed Coordinates: (5, 5, 0)
Unpacked Values -> X: 5, Y: 5, Z: 0
Attempting modification...
Modification Error: 'tuple' object does not support item assignment

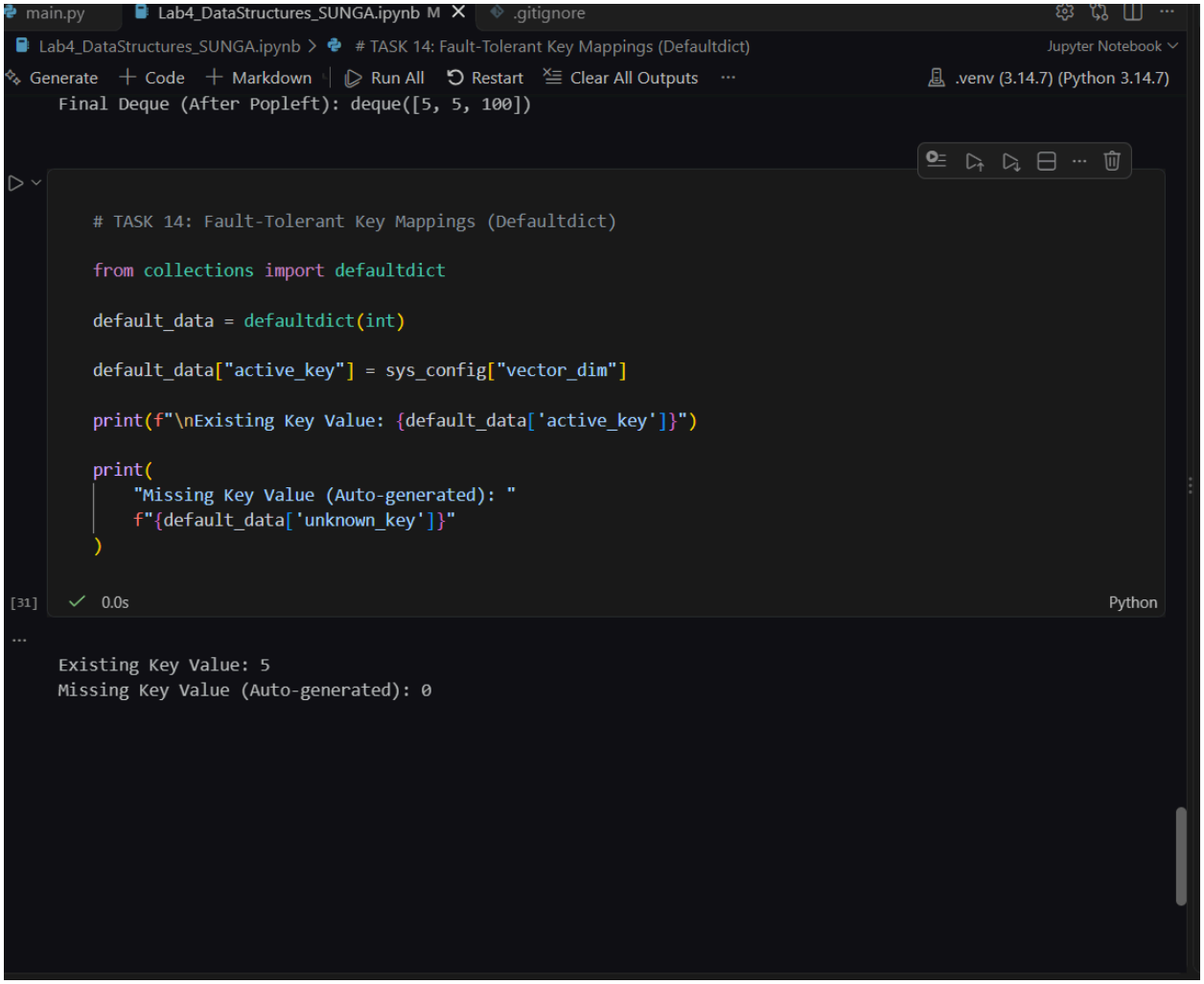

main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
TASK 9: Data De-duplication (Sets)
base_val = sys_config["base_seed"]
raw_data = [
base_val,
base_val + 5,
base_val,
sys_config["vector_dim"],
base_val + 5
]
print(f"\nRaw List (with duplicates): {raw_data}")
unique_data = set(raw_data)
print(f"Unique Set: {unique_data}")
[26]
0.0s
Python
Raw List (with duplicates): [5, 10, 5, 5, 10]
Unique Set: {10, 5}
TASK 10: Mathematical Set Operations
reference_set = {
base_val,
base_val + 10,
base_val + 20
}
[27]
0.0s
Python


main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
TASK 10: Mathematical Set Operations
reference_set = {
base_val,
base_val + 10,
base_val + 20
}
print(f"Reference Set: {reference_set}")
common_elements = unique_data.intersection(reference_set)
combined_elements = unique_data.union(reference_set)
print(f"Intersection (Common): {common_elements}")
print(f"Union (Combined): {combined_elements}")
[27]
0.0s
Python
Reference Set: {25, 5, 15}
Intersection (Common): {5}
Union (Combined): {5, 25, 10, 15}
TASK 11: Homogeneous Arrays
import array
number_array = array.array(
'i',
[sys_config["base_seed"], sys_config["vector_dim"], 100]
)
[28]
0.0s
Python


main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
TASK 11: Homogeneous Arrays
import array
number_array = array.array(
'i',
[sys_config["base_seed"], sys_config["vector_dim"], 100]
)
print(f"\nInteger Array: {number_array}")
print("\nAttempting to append a string...")
try:
number_array.append("invalid")
except TypeError as error_msg:
print(f"Type Error: {error_msg}")
[28]
0.0s
Python
Integer Array: array('i', [5, 5, 100])
Attempting to append a string...
Type Error: 'str' object cannot be interpreted as an integer
TASK 12: List Comprehensions
base_val = sys_config["base_seed"]
generated_list = [
[29]
0.0s
Python


main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
TASK 12: List Comprehensions
base_val = sys_config["base_seed"]
generated_list = [
x * base_val for x in range(1, 6)
]
print(f"\nGenerated Comprehension: {generated_list}")
filtered_list = [
x for x in generated_list if x > 15
]
print(
"Filtered Comprehension (Values > 15): "
f"{filtered_list}"
)
[29]
0.0s
Python
Generated Comprehension: [5, 10, 15, 20, 25]
Filtered Comprehension (Values > 15): [20, 25]
TASK 13: Double-Ended Queues (Deque)
from collections import deque
[30]
0.0s
Python


main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
TASK 13: Double-Ended Queues (Deque)
from collections import deque
data_queue = deque([
sys_config["base_seed"],
sys_config["vector_dim"]
])
print(f"\nInitial Deque: {data_queue}")
data_queue.append(100)
data_queue.appendleft(200)
print(f"After Appends: {data_queue}")
data_queue.popleft()
print(f"Final Deque (After Popleft): {data_queue}")
[30]
0.0s
Python
Initial Deque: deque([5, 5])
After Appends: deque([200, 5, 5, 100])
Final Deque (After Popleft): deque([5, 5, 100])
TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
from collections import defaultdict
[31]
0.0s
Python


main.py
Lab4_DataStructures_SUNGA.ipynb M
.gitignore
Lab4_DataStructures_SUNGA.ipynb > # TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
.venv (3.14.7) (Python 3.14.7)
Final Deque (After Popleft): deque([5, 5, 100])
TASK 14: Fault-Tolerant Key Mappings (Defaultdict)
from collections import defaultdict
default_data = defaultdict(int)
default_data["active_key"] = sys_config["vector_dim"]
print(f"\nExisting Key Value: {default_data['active_key']}")
print(
"Missing Key Value (Auto-generated): "
f"{default_data['unknown_key']}"
)
[31]
0.0s
Python
Existing Key Value: 5
Missing Key Value (Auto-generated): 0